

# Post-Hospital Discharge Risk Recommendation System

Srinija Jonnavithula  
M.S. in Applied Data Science  
University of Florida

**Abstract**—Hospital readmissions are an important quality metric in healthcare and are formally monitored under the Centers for Medicare and Medicaid Services (CMS) Hospital Readmissions Reduction Program (HRRP). This project developed an end-to-end machine learning system to assess hospital-level readmission risk and provide actionable recommendations. CMS HRRP hospital-level data was cleaned, aggregated, and transformed into engineered features that captured readmission performance, variability, discharge volume, measure coverage, and stability. Multiple models, including Logistic Regression, Random Forest, Gradient Boosting, and XGBoost, were trained and compared. All models achieved strong predictive performance, with ROC-AUC values ranging from approximately 0.94 to 0.95. A tuned Random Forest model was selected as the final model because it demonstrated the strongest cross-validation stability and practical generalization. The final system was deployed as an interactive Streamlit dashboard that supports hospital selection, manual input, risk assessment, explanation, and recommendation generation. The results showed that feature engineering contributed more to predictive performance than model complexity and that integrating interpretability and recommendation logic improved the practical value of the system.

## I. PROBLEM STATEMENT

Hospital readmissions are widely used as an indicator of healthcare quality because repeated admissions can reflect gaps in discharge planning, follow-up care, patient transition management, or broader operational coordination. Under the CMS Hospital Readmissions Reduction Program (HRRP), hospitals may face penalties when readmission performance exceeds expected, risk-adjusted benchmarks. This creates both a financial and clinical motivation to identify hospitals that are likely to be at elevated readmission risk.

Many predictive modeling pipelines focus primarily on classification accuracy and stop at the stage of determining whether a hospital is high risk or low risk. However, in applied healthcare settings, prediction alone is often insufficient. A practical system should also help users understand why a hospital is being flagged and what categories of actions may help reduce that risk. Therefore, the final scope of this project focused on building a hospital-level readmission risk recommendation system that combined three components: (1) predictive modeling, (2) interpretable explanations, and (3) actionable recommendations delivered through an interactive dashboard.

## II. RELATED WORK

Readmission prediction has been studied extensively in clinical informatics and healthcare analytics. Logistic Regression has long served as a standard baseline because it is interpretable and performs well on structured tabular datasets. Ensemble methods such as Random Forest and Gradient Boosting are commonly used because they capture non-linear interactions among predictors and often outperform purely linear models in real-world operational datasets.

In the broader machine learning ecosystem, Scikit-learn provides standardized implementations for Logistic Regression, Random Forest, and Gradient Boosting, while XGBoost provides an optimized boosting framework for structured classification tasks. Streamlit is increasingly used for rapid deployment of decision-support applications due to its simple integration with trained model artifacts.

Despite the extensive literature on readmission prediction, many systems remain prediction-oriented and do not include interpretable recommendation outputs. This project differentiated itself by combining machine learning risk assessment with a feature-driven recommendation layer and integrating the complete workflow into a user-facing dashboard.

## III. OBJECTIVES

The final objectives of the project were:

- Develop a hospital-level machine learning framework for readmission risk classification.
- Engineer meaningful hospital-level predictors from CMS HRRP data.
- Compare baseline and advanced machine learning models.
- Select a stable and generalizable final model.
- Build an interactive dashboard for risk assessment.
- Generate feature-driven recommendations for improvement.

These objectives were achieved. Compared with the original concept, the final implementation placed stronger emphasis on interpretability and practical decision support rather than on maximizing model complexity.

## IV. FINAL DELIVERABLES

The final system included the following deliverables:

- A cleaned and processed hospital-level dataset derived from CMS HRRP data.

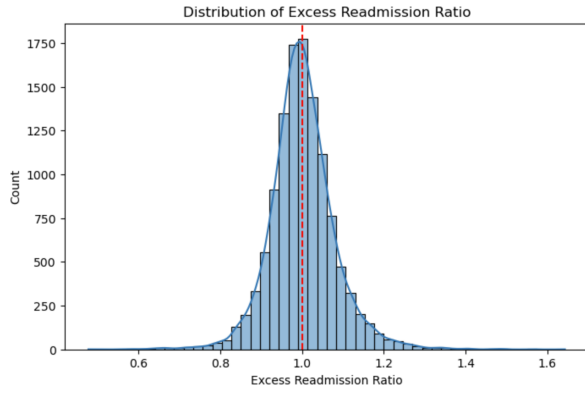


Fig. 1: Example feature distribution used to inspect class boundary behavior.

- A complete feature engineering pipeline implemented in Jupyter notebooks.
- Baseline and advanced machine learning models for risk classification.
- A tuned Random Forest final model saved as a reusable model artifact.
- A Streamlit dashboard for interactive risk assessment.
- A rule-based recommendation engine linked to feature patterns.
- A public GitHub repository with notebooks, dashboard code, README, and requirements.
- A poster, presentation slides, and a video demonstration.

## V. EXPLORATORY DATA ANALYSIS

Exploratory Data Analysis (EDA) was conducted before model development to validate the structure, quality, and predictive suitability of the dataset.

### A. Dataset Structure

The dataset was sourced from the CMS Hospital Readmissions Reduction Program and contained hospital-level readmission-related performance metrics across multiple reported conditions. After aggregation to hospital level, the resulting summary dataset contained 2862 hospital records before handling the remaining missing target-related values. After excluding hospitals without valid volume-weighted excess readmission ratio values, the modeling dataset contained 2496 hospitals.

### B. Feature Distributions

Distribution analysis showed that many hospitals clustered near the benchmark threshold of  $ERR = 1.0$ . This was expected because hospitals tend to concentrate near the CMS performance boundary rather than being strongly separated into extreme low-risk and high-risk groups. This observation suggested that borderline hospitals would be more difficult to classify correctly than hospitals with clearly elevated or clearly stable readmission performance.

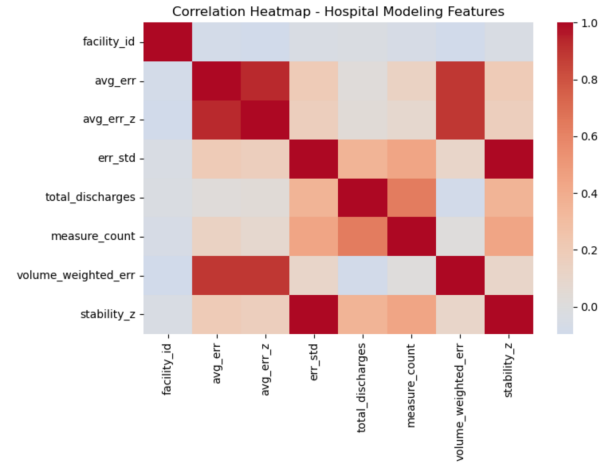


Fig. 2: Correlation heatmap of engineered hospital-level features.

### C. Correlation Analysis

A correlation heatmap was generated to evaluate relationships among engineered hospital-level features. Strong positive correlation was observed between average excess readmission ratio and volume-weighted ERR, which confirmed that the engineered summary features were aligned with the target concept. At the same time, additional features such as variability, discharge volume, and stability captured complementary information.

### D. EDA Insights

The EDA phase yielded the following key insights:

- Average readmission performance was a dominant signal in the data.
- Variability and stability features contributed additional structure beyond overall averages.
- Hospitals near the benchmark threshold were expected to be harder to classify.
- The dataset contained enough signal for supervised learning without requiring additional augmentation.

## VI. METHODOLOGY

### A. Data Processing

The raw CMS HRRP dataset was cleaned and standardized before modeling. Column names were normalized, numeric fields were converted to valid numeric format, and records without usable readmission metrics were excluded. The modeling population was restricted to hospitals with valid excess readmission ratio measurements and associated discharge information.

### B. Feature Engineering

The project used hospital-level engineered features rather than raw row-level variables. The final features were:

- **avg\_err**: mean excess readmission ratio across measures. This captures the overall level of readmission-related performance.

- **avg\_err\_z**: standardized form of average ERR. This provides normalized context relative to the broader dataset.
- **err\_std**: standard deviation of ERR across measures. This captures inconsistency and condition-level variability.
- **total\_discharges**: total discharge volume. This represents hospital operational scale and patient throughput.
- **measure\_count**: number of HRRP-related measures reported. This reflects the breadth of hospital evaluation.
- **stability\_z**: a standardized stability metric. This captures whether performance appears relatively stable or unstable.

These features were chosen because they jointly represent performance magnitude, spread, breadth of reporting, and operational burden.

### C. Target Variable

The target variable was defined as hospital-level risk status:

$$Y = \begin{cases} 1, & \text{if volume-weighted ERR} > 1.0 \\ 0, & \text{otherwise} \end{cases} \quad (1)$$

This threshold was chosen because ERR values greater than 1.0 indicate excess readmissions relative to the expected CMS benchmark. The use of a volume-weighted form improves realism by accounting for hospital discharge scale.

### D. Feature Representation

Each hospital was represented as a feature vector:

$$X = [\text{avg\_err}, \text{avg\_err\_z}, \text{err\_std}, \text{total\_discharges}, \text{measure\_count}, \text{stability\_z}] \quad (2)$$

The learning task was to estimate a mapping:

$$f(X) \rightarrow Y \quad (3)$$

where  $Y \in \{0, 1\}$  denotes low-risk or high-risk classification.

### E. Models Evaluated

The following models were trained and compared:

- Logistic Regression
- Random Forest (base)
- Random Forest (tuned)
- Gradient Boosting
- XGBoost

Logistic Regression served as the baseline because it is interpretable and often effective for structured data. Ensemble methods were included to evaluate whether more complex non-linear learners substantially improved predictive performance.

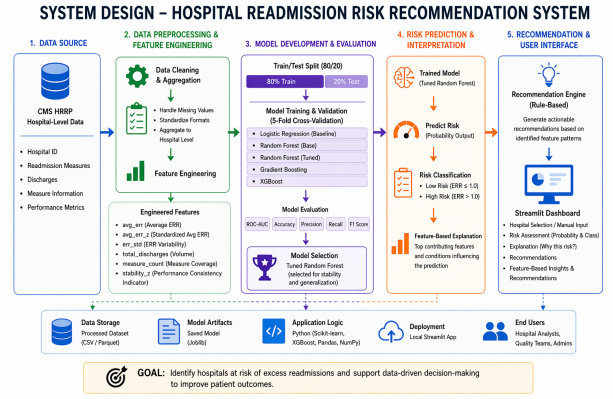


Fig. 3: System architecture of the risk recommendation pipeline.

### F. Random Forest Rationale

Random Forest was an especially strong candidate for this project because it handles feature interactions well, is robust to non-linear relationships, and provides intuitive feature importance estimates. The ensemble prediction can be expressed as:

$$\hat{Y} = \frac{1}{N} \sum_{i=1}^N T_i(X) \quad (4)$$

where  $T_i$  denotes the prediction of the  $i$ th decision tree and  $N$  is the number of trees in the forest.

### G. Hyperparameter Tuning

A tuned Random Forest model was developed using grid search. The best-performing configuration used:

- `n_estimators = 200`
- `max_depth = 10`
- `min_samples_leaf = 4`
- `min_samples_split = 2`

This tuned model was retained as the final model because it provided stronger cross-validation performance and stability.

## VII. SYSTEM DESIGN

The final system followed a modular architecture that connected data preparation, model inference, explanation, and recommendation logic through a user-facing dashboard.

The architecture begins with raw CMS HRRP data. After preprocessing and feature engineering, the resulting hospital-level feature vectors are used to train and evaluate multiple machine learning models. The selected final model outputs a risk probability and a binary classification. This output is then passed into an explanation layer and a recommendation engine. Finally, the entire workflow is surfaced through a Streamlit dashboard.

TABLE I: Test-set performance comparison

| Model                 | ROC-AUC | Accuracy | F1 Score |
|-----------------------|---------|----------|----------|
| Random Forest (Base)  | 0.9466  | 0.878    | 0.8885   |
| Random Forest (Tuned) | 0.9465  | 0.876    | 0.8860   |
| Gradient Boosting     | 0.9424  | 0.874    | 0.8873   |
| XGBoost               | 0.9411  | 0.864    | 0.8764   |
| Logistic Regression   | 0.9397  | 0.870    | 0.8825   |

#### A. Dashboard Interface

The dashboard serves as the main access point for the final system. It supports two usage modes:

- selecting an existing hospital from the processed dataset
- entering feature values manually for simulation

The system then performs risk assessment, explains the resulting risk level, and displays feature-driven recommendations.

### VIII. EXPERIMENTAL DESIGN

The experiments were designed to compare baseline and advanced classification models under a consistent evaluation protocol.

#### A. Train/Test Strategy

The dataset was split into training and testing sets using an 80/20 partition. This ensured that the final reported metrics were computed on unseen test data.

#### B. Cross-Validation

To evaluate generalization, 5-fold stratified cross-validation was applied. Cross-validation was important because single split performance alone can overstate model quality or mask instability.

#### C. Baselines and Comparisons

Logistic Regression was used as the baseline model because it provides a strong interpretable benchmark. Random Forest, Gradient Boosting, and XGBoost were used as ensemble comparisons to determine whether more complex models provide meaningful gains over the baseline.

#### D. Evaluation Metrics

Performance was evaluated using:

- ROC-AUC
- Balanced Accuracy
- Accuracy
- Precision
- Recall
- F1 Score

Cross-validation mean ROC-AUC and standard deviation were also used to select the final model based on stability.

### IX. RESULTS AND ANALYSIS

#### A. Test-Set Model Performance

Table I summarizes the final test-set performance across the evaluated models.

TABLE II: Cross-validation performance comparison

| Model                 | CV Mean ROC-AUC | CV Std ROC-AUC |
|-----------------------|-----------------|----------------|
| Logistic Regression   | 0.9366          | 0.0051         |
| Random Forest (Tuned) | 0.9441          | 0.0046         |
| Gradient Boosting     | 0.9402          | 0.0051         |
| XGBoost               | 0.9422          | 0.0049         |

#### B. Cross-Validation Performance

Table II reports cross-validation performance. This was the main basis for final model selection.

#### C. Performance Interpretation

All models performed strongly, with ROC-AUC values ranging from approximately 0.94 to 0.95. The small gap across models suggests that the dataset is well-structured and that the engineered features already capture most of the predictive signal. In practical terms, this means that the benefit of more complex modeling was modest.

Although the base Random Forest had the highest single test ROC-AUC, the tuned Random Forest achieved the strongest cross-validation mean and the lowest variance among the non-linear models. For that reason, the tuned Random Forest was selected as the final model.

#### D. Analytical Insight

The small differences across models imply that:

- the task is partially linearly separable
- the engineered hospital-level features are highly informative
- model complexity offers limited gains beyond a certain point

This was one of the most important findings of the project: feature engineering contributed more strongly to success than algorithmic complexity alone.

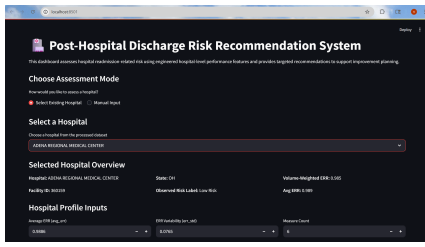
### X. INTERPRETABILITY AND ERROR ANALYSIS

#### A. Interpretability

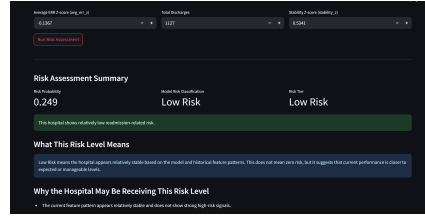
The final system provides interpretability at two levels. First, model-side interpretability was examined using feature importance for the tuned Random Forest. Second, user-facing interpretability was implemented directly in the dashboard through natural language explanation of the contributing feature conditions.

The dashboard explains risk through patterns such as:

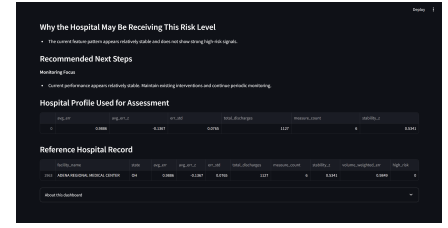
- high average ERR
- elevated variability
- high discharge volume
- low measure coverage
- unstable performance



(a) Hospital selection and feature input



(b) Risk assessment summary



(c) Explanation and recommendation output

Fig. 4: Dashboard workflow from hospital input to risk assessment and recommendations.

## B. Recommendation Logic

The recommendation engine is feature-driven and rule-based rather than fixed by risk class alone. For example:

- high avg\_err triggers discharge planning recommendations
- high err\_std triggers workflow standardization recommendations
- high total\_discharges triggers care coordination recommendations

This means that two hospitals with the same risk tier may receive different recommendations depending on what specific features are driving their classification.

## C. Error Analysis

Most likely misclassifications occur near the decision boundary, especially for hospitals with ERR values close to the CMS benchmark threshold of 1.0. These hospitals can share overlapping feature distributions, making them difficult to separate cleanly.

Additional sources of error include:

- the use of hospital-level aggregate data rather than patient-level records
- overlap in moderate-risk and borderline-risk feature profiles
- absence of clinical detail beyond the available HRRP variables

Thus, the system is strongest when identifying hospitals with clearly elevated or clearly stable performance, while borderline hospitals remain more ambiguous.

## D. Why the System Still Adds Value

Even when classification is uncertain near the boundary, the system remains useful because it does more than output a label. It provides:

- a probability score
- a risk tier
- explanation of likely contributing factors
- actionable recommendation categories

This makes it more useful than a black-box classifier from an operational perspective.

## XI. CHALLENGES AND LESSONS

### A. Challenge 1: Similar Model Performance

Because all models performed well and similarly, model selection was not straightforward. This challenge was addressed by prioritizing cross-validation stability instead of using test-set ROC-AUC alone.

### B. Challenge 2: Recommendation Design

The recommendation system was challenging because the dataset did not include intervention outcome labels. This was addressed by creating a feature-driven rule-based mapping that linked measurable hospital patterns to reasonable operational suggestions.

### C. Challenge 3: Dashboard Usability

The dashboard required multiple iterations to ensure that the distinction between risk assessment, explanation, and recommendation was clear. Several wording changes were made so that the system communicated assessment rather than only prediction.

### D. Lessons Learned

The project reinforced several practical lessons:

- engineered features are crucial in structured healthcare datasets
- strong predictive performance does not necessarily require highly complex models
- interpretability and usability are essential for real-world adoption

## XII. CONCLUSION

This project developed an end-to-end hospital readmission risk recommendation system using CMS HRRP hospital-level data. Multiple machine learning models were evaluated, and a tuned Random Forest was selected as the final model because of its strong cross-validation stability and generalization. The final system combined prediction, explanation, and feature-driven recommendations within an interactive Streamlit dashboard.

The main technical conclusion is that strong feature engineering enabled high predictive performance across all evaluated models and that model complexity offered only marginal improvements. The main applied conclusion is that integrating

recommendations and interpretability transformed the system from a classification pipeline into a decision-support tool.

Future researchers or developers could extend this work by:

- incorporating patient-level data for more granular modeling
- replacing the rule-based recommendation engine with data-driven recommendation learning
- deploying the system in a containerized or cloud-native environment

### XIII. APPENDICES

GitHub Repository:

Post-Hospital Discharge Risk Recommendation System  
GitHub Repository

Project Video and Materials:

Google Drive Folder for Video, Poster, and Presentation Materials

### REFERENCES

- [1] Centers for Medicare and Medicaid Services, “Hospital Readmissions Reduction Program (HRRP),” CMS Public Data.
- [2] F. Pedregosa *et al.*, “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [3] L. Breiman, “Random Forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
- [4] T. Chen and C. Guestrin, “XGBoost: A Scalable Tree Boosting System,” in *Proc. KDD*, 2016, pp. 785–794.
- [5] Streamlit, “Streamlit: The Fastest Way to Build Data Apps,” 2026.